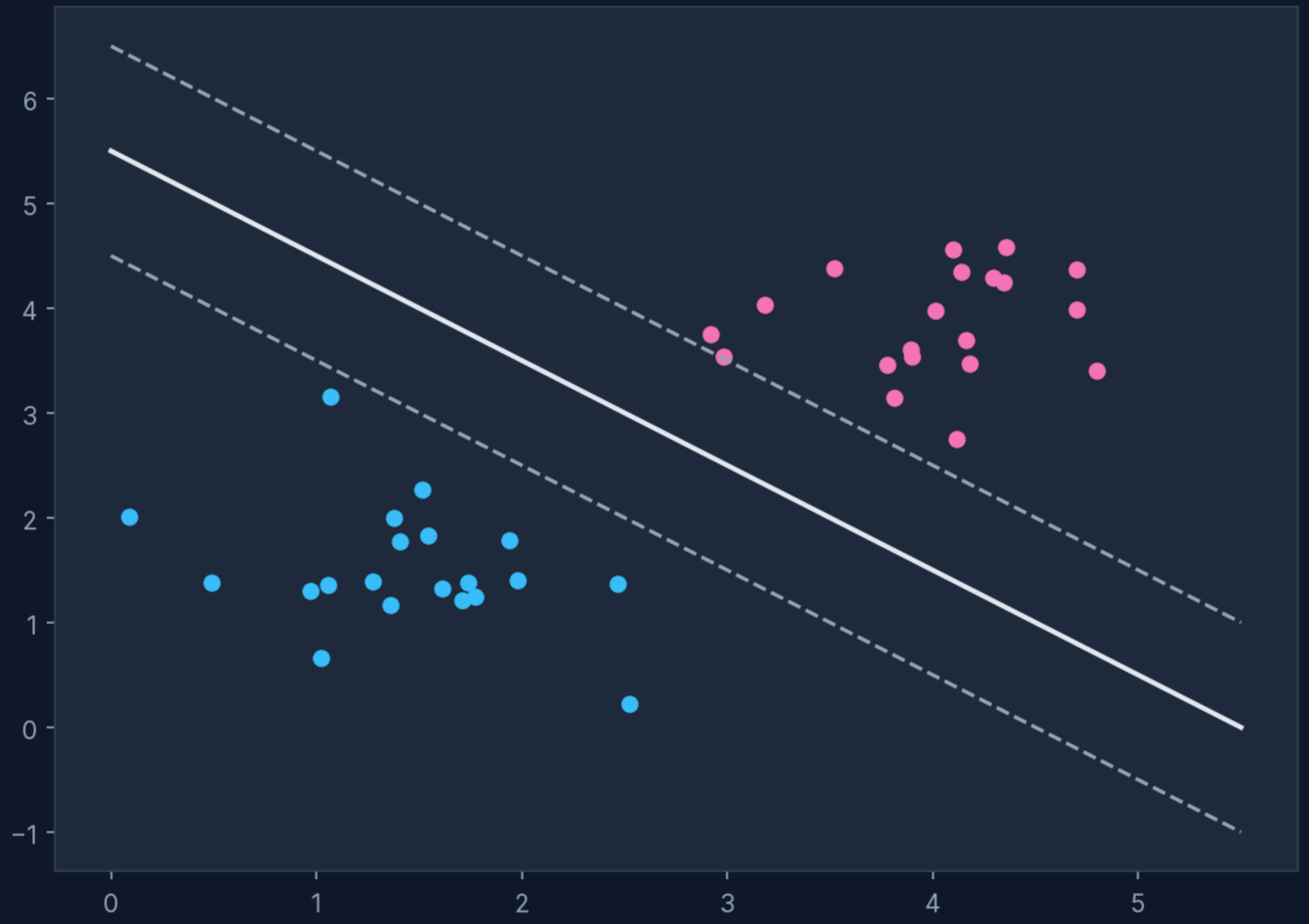
Support Vector Machines

Maximum-margin classifiers

The problem

- Two classes, many separating lines
- Which one generalises best?
- Must tolerate a few overlapping points

Intuition: widest street



Pick the boundary with max mar

Only points on the edge matter:

the support vectors

Hinge loss objective

$$\min_{w} \ \frac{\lambda}{2}\|w\|^2 + \frac{1}{n}\sum_{i}\max(0, 1 - y_i\, w^\top x_i)$$

- Small ‖w‖ ⇔ wide margin
- Loss is zero once a point is beyond the margin
- Labels are −1 / +1

Pegasos: stochastic sub-gradient

- Step size $\eta_t = 1 / (\lambda\, t)$
- Always shrink: $w \leftarrow (1 - \eta\lambda)\, w$
- If margin < 1: $w \leftarrow w + \eta\, y\, x$
- Bias folded in as an extra feature

LinearSVM.fit

```
def fit(self, X, y):
    X = np.asarray(X, float)
    y = np.where(np.asarray(y) > 0, 1.0, -1.0)
    self.classes_ = (0, 1)
    Xb = np.hstack([X, np.ones((len(X), 1))])  # bias as extra feature
    w = np.zeros(Xb.shape[1])
    rng, t = np.random.default_rng(self.seed), 0
    for _ in range(self.epochs):
        for i in rng.permutation(len(X)):
            t += 1
            eta = 1.0 / (self.lam * t)
            margin = y[i] * Xb[i] @ w
            w *= 1 - eta * self.lam
            if margin < 1:
                w += eta * y[i] * Xb[i]
    self.w, self.b = w[:-1], w[-1]
    return self
```

Complexity & trade-offs

- Pegasos: O(epochs · n · d), independent of margin
- Robust: only support vectors shape the boundary
- Non-linear: kernel trick (RBF, polynomial)
- No native probabilities; scale features

Recap

- Maximise the margin
- Hinge loss + L2 penalty
- Pegasos: shrink, then fix violations
- Kernels for non-linear data